Data Visualization and Exploration

COMS7056A/COMS4060A

Muhammad Nasir

Lecture 4

Lesson Plan

- Feature selection:
  - Filter-based methods
  - Wrapper methods
  - Embedded methods
- Key references:
  - Data Mining 3rd Edition, Chapter 7, by I. Witten, E. Frank, & M. Hall
  - Feature Engineering for Machine Learning and Data Analytics, Chapter 8, by G. Dong & H. Liu
  - Applied Predictive Modeling, Chapters 18 and 19, by K. Johnson and M. Kuhn,

Motivation

- Better data beats fancier algorithms.
- **Feature engineering:** We create new features from raw data to increase the predictive power of our final model. These new features should capture extra information that is not obvious in the original features.
- **Feature selection:** The process of selecting the most informative subset of features and reduce the dimensionality of the problem.

Feature Selection

- Isn’t more features better?
- In theory: yes! In practice: adding irrelevant or distracting attributes to a dataset often confuses the machine learning systems. – Data Mining 3rd Edition (I. Witten et. al.)
- Even relevant features can hurt.
- Pre-selecting can often be helpful. Especially when we have domain knowledge.

Types Of Features

- Irrelevant features
- Redundant features
- Weakly relevant but non-redundant features
- Strongly relevant features
- Our goal is to find the strongly relevant features!

Method types

- Search-based:
  - **Wrapper Methods:** Search for well-performing subsets of features.
    - A base model/learning algorithm is “wrapped” into the selection procedure
    - A feature set is identified by searching through many different subsets, training the base learning, and evaluating the model on some metric (AIC, BIC for example)
    - The search process for choosing feature subsets can be random, or iteratively add or remove features one at a time.
  - **Filter methods:** Select subsets of features based on their relationship with the target.
    - The relationship between the target and feature is given a score, through statistical methods and feature importance methods based on entropy, correlation-based, inter-class distance, mutual information.
    - No actual learning/model is built until the attributes are selected
  - **Intrinsic/embedded methods:** Algorithms that perform automatic feature selection during training.
- Embedded feature selection
  - Decision trees, auto-encoders, LASSO, for example
- Correlation-based (mRMR) using relevance and redundancy analysis
- Some texts will also refer to supervised vs unsupervised, or sparse vs dense, but the above methods all overlap with these in some way

Wrapper Methods

- These are methods ”wrapped” into the training.
- Train on a subset and evaluate performance (AIC, BIC etc). Try another subset, repeat. Use the best subset.
- Does a good job of getting rid of correlated pairs.
- Examples are forward selection, backwards selection, exhaustive etc.
- For these we need:
    - A base model/learner usually a simple model, but really there isn’t a restriction on this. Linear or logistic regression algorithms, or Random Forest are a good start
    - Search procedure
    - Objective function

Wrapper Methods

```
1  Create an initial model containing only an intercept term.
2  repeat
3      for each predictor not in the current model do
4          Create a candidate model by adding the predictor to the
           current model
5          Use a hypothesis test to estimate the statistical significance
           of the new model term
6      end
7      if the smallest p-value is less than the inclusion threshold then
8          Update the current model to include a term corresponding to
           the most statistically significant predictor
9      else
10         Stop
11     end
12 until no statistically significant predictors remain outside the model
```

Applied Predictive Modeling, Algorithm 19.1, pg. 491.

Search Procedures

- Forward selection:
  - The initial model has no features, and features are added one at a time, provided the feature provides suitable performance for the model
- Backward selection
  - The initial model includes all features and remove features one at a time. The feature removed is the one that makes the biggest improvement in the model performance. The stopping criteria is the same as for forward selection.
- Exhaustive:
  - Use every single combination of features. This is unlikely to be computationally feasible.
- Hybrid approach:
  - Combination of a greedy algorithm and a filter method.
- Recursive Feature Elimination (RFE):
  - A model is fitted, and the weakest feature (or features) gets removed until the specified number of features is reached.
  - Specifically, this model should assign weights or feature importances to variables so that these can be used to identify the weakest features
  - See Sklearn RFE for a Python implementation

Forward/backward search

- Notice how performing the search in forward and backward produces different results! This is something to be aware of, and a reminder to perform crossvalidation to check your features!
- Note that you can set the number of features, the number of cross validation folds to use, and the scoring metric

Forward

```
#FOR CLASSIFICATION MODEL
dataset = 'wine' # 'breast_cancer'
k = 3
cv = 3
scoring='accuracy'
feature_select = SequentialFeatureSelector(LogisticRegression(multi_class='ovr', solver='newton-cg'),
                                           k_features=k,
                                           forward=True,
                                           floating=False,
                                           scoring=scoring,
                                           cv=cv)
feature_select.fit(ds[dataset]['x_train'], ds[dataset]['y_train'])
feature_select.k_feature_names_
```

✓ 1.9s

```
('alcohol', 'ash', 'flavanoids')
```

Backward

```
#FOR CLASSIFICATION MODEL
dataset = 'wine'
k = 3
cv = 3
scoring = 'accuracy'
feature_select = SequentialFeatureSelector(LogisticRegression(solver='newton-cg'),
                                           k_features=k,
                                           forward=False,
                                           floating=False,
                                           scoring=scoring,
                                           cv=cv)
feature_select.fit(ds[dataset]['x_train'], ds[dataset]['y_train'])
feature_select.k_feature_names_
```

✓ 12.9s

```
('flavanoids', 'color_intensity', 'proline')
```

Optimality Criteria

- Choosing appropriate criteria is difficult as there are multiple objectives in feature selection. Often, the criteria measures accuracy, penalised by the number of features selected. Usually you can set the optimisation metric in your algorithm as an additional parameter, so you don't need to code any of these.
- Some common options are:
  - R-squared: it is the proportion of variance explained by your model.
  - p-values: In the context of regression where you're trying to estimate coefficients to think in terms of p-values, you make an assumption of there being a null hypothesis that the coefficients are zero. For any given correlation coefficient, the p-value captures the probability of observing the data that you observed, and obtaining the test-statistic (in this case the estimated correlation coefficient) that you got under the null hypothesis. If you have a low p-value, it is highly unlikely that you would observe such a test-statistic if the null hypothesis actually held. This translates to meaning that (with some confidence) the coefficient is highly likely to be non-zero.
  - AIC (Akaike Information Criterion)
  - BIC (Bayesian Information Criterion)

Filter-Based Feature Selection

- Filter feature selection methods use statistical techniques to evaluate the relationship between each input feature and the target variable, providing a score, which is used as a basis to choose (filter) those input variables that will be used in the model
- For regression: ANOVA, mutual information
- For classification: chi-squared, F-regression, mutual information
- These are typically univariate methods, working only between a single feature and the target variable, and do not consider relationships or interactions between variables. It’s easy to pick too many variables because we are not necessarily aware of the relationship or dependence between two or more features
- However, filter-based methods are usually much faster (and less computationally expensive) than wrapper methods

Simple Methods: Getting A Baseline

- Variance thresholds:
  - The variance threshold is a simple technique. We set a variance threshold, and remove all features that do not meet this. The idea is that features with a higher variance are more useful – they have more information. However, this ignores any interactions between variables!
- Threshold on correlation coefficient:
  - A number of the filter-based methods use correlation coefficients for selection. In general, we use this to create a test statistic, and check if it is a statistically significant feature. However, we can use an absolute value too as a baseline. Additionally, we can use this method to drop mutually correlated features.

Chi Squared test

- The test measures dependence between variables, using this finds the features that are the most likely to be independent of the target category, and therefore irrelevant for classification purposes.
- Then, we want to compare what is observed in the dataset, with what we expect to find if the categories are independent from each other
  - The expected values are calculated according to:
    - Expected = (total in row * total in column) / total
    - Eg. Female, 1st class: (94+76+144) * (94 + 122) / 891 = 76
  - The chi squared test assumes a null hypothesis that the observed match the expected, ie, all variables are independent
- Chi squared stat is calculated by: $x^2 = \sum_{i=1}^{k} \sum_{j=1}^{m} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$
- Finally, we choose n features with the highest test statistics or lower p-value

[22]:
```
cont = pd.crosstab(df.Sex, df.Pclass)
print(cont)
```

```
Pclass    1    2    3
Sex
female   94   76  144
male    122  108  347
```

[23]:
```
stat, p, dof, expected = chi2_contingency(cont)
print(expected)
```

```
[[ 76.12121212  64.84399551 173.03479237]
 [139.87878788 119.15600449 317.96520763]]
```

[43]:
```
chi2_tab = (cont - expected)**2/expected
print(chi2_tab)
```

```
Pclass         1         2         3
Sex
female  4.199238  1.919321  4.871963
male    2.285200  1.044483  2.651294
```

[45]:
```
print('computed chi2', chi2_tab.sum().sum())
print('chi2_contingency chi2', stat)
```

```
computed chi2 16.97149909551711
chi2_contingency chi2 16.971499095517114
```

[50]:
```
# is it stat significant at 95% level?
critical_value = chi2.ppf(0.95, dof)
signif = abs(stat)>critical_value
print('chi2', stat)
if signif:
    print('Reject H0, dependent')
else:
    print('Cannot reject H0')

# Use the p-value instead
alpha = 1.0 - 0.95
print('p-value:', p)
if p <= alpha:
    print('Reject H0, dependent')
else:
    print('Cannot reject H0')
```

```
chi2 16.971499095517114
Reject H0, dependent
p-value: 0.00020638864348233114
Reject H0, dependent
```

Feature Importances

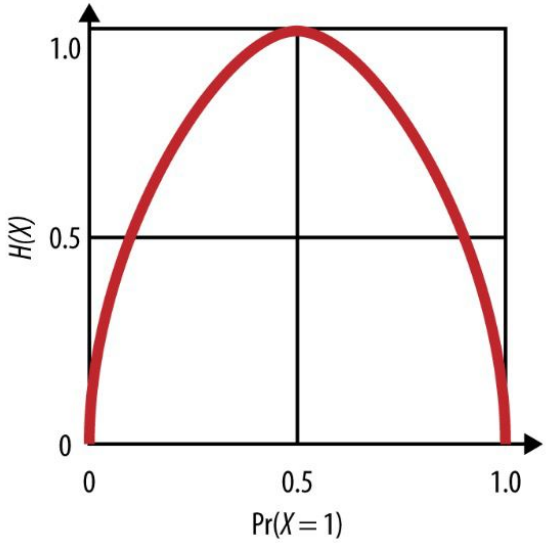
- Feature importance is a quantification of the relationship between features and a target variable. Different methods are required if the target or features are numerical or categorical.
- Numerical target
  - Numerical input:
    - Here it's appropriate to use Pearson's or Spearman's correlation coefficients and then testing to see if they are statistically significant
  - Categorical input:
    - We can look at the mean target value for each category and perform a t-test (if normally distributed, otherwise try Wilcoxon rank sum) to see if they are statistically different
- Categorical target:
  - Numerical input:
    - A simple method is to use the mean values of the input features for each target class and perform a t-test on these to check if they are different
  - Categorical input:
    - We can look at information gain
    - Or Fisher's exact test or odds ratio if there are only 2 classes

Entropy

- Entropy refers to the expected uncertainty in a variable and can be used to find out how much information might be gained from knowing this variable
  - Entropy, for a target variable X, is given by: $H(X) = -\sum_{x} P(x)log(P(x))$ where X comes from a pmf of P(x)
    - H(x) gives us the expected uncertainty in X, or, how much information is learnt on average from X
  - If two variables, X and Y, are jointly distributed according to P(x,y), the joint entropy is given by: $H(X,Y) = -\sum_{x,y} P(x,y)log(P(x,y))$
  - The conditional entropy tells us how much uncertainty there is in X if we know Y: $H(X|Y) = -\sum_{x,y} P(x,y)log(P(x|y))$

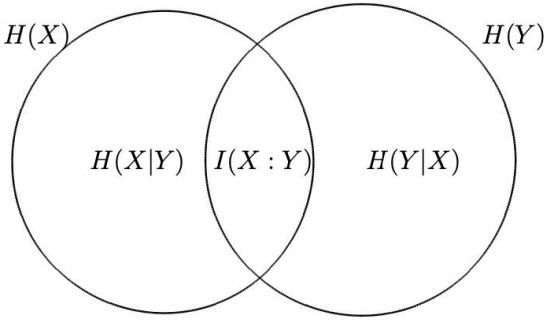
Entropy

- Practically, having a skewed probability distribution indicates low entropy, and a balanced distribution indicates high entropy
  - If you know there is a 50% chance (eg coin toss), then the entropy is 1 (maximum)
  - If you know that there is 10% chance of an event happening, then the entropy will be lower (there is less information gained by that event): we can use entropy to define the balance of a dataset too



Mutual Information

- Mutual information is the concept that variables will contain information about each other if there is any sort of dependence between them (linear or not, numerical or categorical)
  - Given two features, X and Y, mutual information is used to quantify how much knowing the value of Y will reduce the uncertainty in X. If they are independent, then knowing X will give us no information about Y, so zero information gain. Similarly, if there is a deterministic relationship between them, the information gain will be maximal.
  - In information theory, entropy refers to uncertainty or the amount of information in a variable, so mutual information can be described in terms of entropy
    - I(X;Y) = H(X) – H(X|Y), ie, how much uncertainty is removed by knowing Y?
- When applying this to feature selection, we typically will calculate the mutual information for all the features, and then choose the top K features
- Note: mutual information and information gain (eg. Decision trees) are used interchangeably



http://www.info612.ece.mcgill.ca/lecture_02.pdf

Mutual Information Regression

Regression

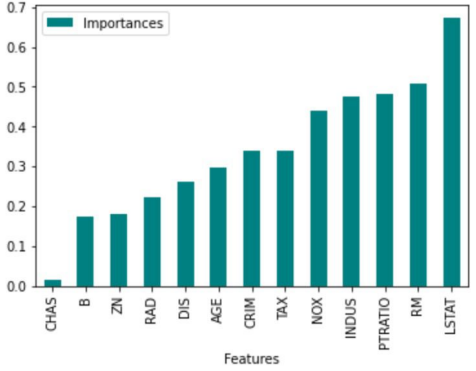
```
dataset = 'boston'

# example of mutual information feature selection for numerical input data
def select_features(X_train, y_train, X_test):
    # configure to select all features
    fs = SelectKBest(score_func=mutual_info_regression, k='all')
    fs.fit(X_train, y_train)
    # transform train input data
    X_train_fs = fs.transform(X_train)
    # transform test input data
    X_test_fs = fs.transform(X_test)
    return X_train_fs, X_test_fs, fs

# If you want to create your own dataset
# X, y = make_regression(n_samples=1000, n_features=100, n_informative=10, noise=0.1, random_state=1)
# split into train and test sets
# X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=1)
# feature selection
_, _, fs = select_features(ds[dataset]['x_train'], ds[dataset]['y_train'],ds[dataset]['x_valid'])

final_df = pd.DataFrame({ "Features": pd.DataFrame(ds[dataset]['x_train']).columns, \
                          "Importances": fs.scores_, "Included": fs.get_support()})
final_df.set_index('Importances')
# sort in ascending order to better visualization.
final_df = final_df.sort_values('Importances')
# plot the feature importances in bars.
final_df.plot.bar(color='teal', x='Features')
# what are scores for the feature
for i in range(len(fs.scores_)):
    print('Feature %d: %f' % (i, fs.scores_[i]))
```

```
Feature 0: 0.338664
Feature 1: 0.179428
Feature 2: 0.476692
Feature 3: 0.014828
Feature 4: 0.438236
Feature 5: 0.508649
Feature 6: 0.297733
Feature 7: 0.261638
Feature 8: 0.223895
Feature 9: 0.339232
Feature 10: 0.480366
Feature 11: 0.174690
Feature 12: 0.671884
```



Mutual Information Classification

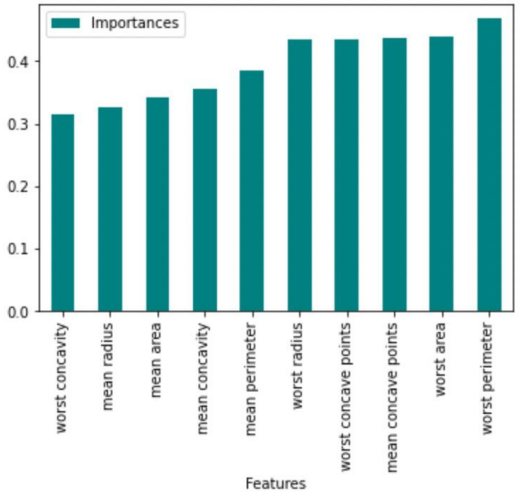
```python
# prepare target
def prepare_targets(y_train, y_test):
    le = LabelEncoder()
    le.fit(y_train)
    y_train_enc = le.transform(y_train)
    y_test_enc = le.transform(y_test)
    return y_train_enc, y_test_enc

def select_features(X_train, y_train, X_test):
    # configure to select 10 features
    fs = SelectKBest(score_func=mutual_info_classif, k=10)
    fs.fit(X_train, y_train)
    # transform train input data
    X_train_fs = fs.transform(X_train)
    # transform test input data
    X_test_fs = fs.transform(X_test)
    return X_train_fs, X_test_fs, fs

# prepare output data
y_train_enc, y_test_enc = prepare_targets(ds[dataset]['y_train'], ds[dataset]['y_valid'])

_, _, fs = select_features(ds[dataset]['x_train'], y_train_enc, ds[dataset]['x_valid'])
# print(fs.get_support())
final_df = pd.DataFrame({ "Features": pd.DataFrame(ds[dataset]['x_train']).columns,
                          "Importances": fs.scores_, "Included": fs.get_support()})
final_df.set_index('Importances')
# sort in ascending order to better visualization.
final_df = final_df[final_df.Included == True].sort_values('Importances')
# plot the feature importances in bars.
final_df.plot.bar(color='teal', x='Features')
# what are scores for the feature
for _, row in final_df.iterrows():
    print('Feature %s: %f' % (row['Features'], row['Importances']))
```

```
Feature worst concavity: 0.315260
Feature mean radius: 0.325420
Feature mean area: 0.341036
Feature mean concavity: 0.354883
Feature mean perimeter: 0.384964
Feature worst radius: 0.433926
Feature worst concave points: 0.434384
Feature mean concave points: 0.435989
Feature worst area: 0.439824
Feature worst perimeter: 0.468578
```



Minimum-Redundancy-Maximum-Relevance (mRMR)

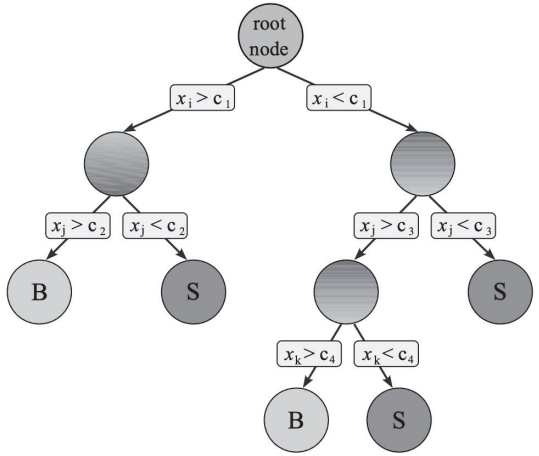
- The mRMR algorithm is designed to identify features that reduce their redundancy in the presence of other features, whilst simultaneously increasing their own relevance:
  - Relevance: this captures the relationship between a feature *f* and a target variable *y*
  - Redundancy: this captures the relationship between a feature *f* and a subset of other features *S*
  - The most common definition is mRMR=relevance – redundancy, although a ratio of the two can also be used.
- The general procedure is like that of the wrapper methods:
  - We are looking for a subset of the full feature set, which we call S. This subset S is built up iteratively, adding a single feature at a time. We loop through all features, adding the one that gives the best mRMR metric. The first feature selected is the one with the greatest relevance (there are no other features in the subset to check the redundancy yet). At each step subsequent step, we evaluate the relevance and redundancy of the candidate feature, and add the best performing feature to our current subset S. We stop once we've reached a predefined number of features, K.
- The actual mathematical definition of relevance and redundancy are configurable based on the problem and the datatypes (numerical or categorical)but they are often defined in terms of mutual information or correlations
- Eg. For mutual information:
  - Relevance: $D(S,y) = \frac{1}{|S|} \sum_{f_i \in S} I(f_i;y)$, Redundancy: $R(S) = \frac{1}{|S|^2} \sum_{f_i,f_j \in S} I(f_i;f_j)$
- For correlation:
  - Relevance: $F = \frac{corr^2}{1-corr^2} * dof$, where you can use can use Pearson's or Spearman's correlation coefficient ,and *dof* degrees of freedom (size(y) - 1). Redundancy: $R(S) = \sum_{f_j \in S} \frac{|corr(f_j,y)|}{i-1}$

Embedded Methods

- The features are selected during the training.
- Ridge, LASSO , auto-encoders, decision trees.
- Generally they’re faster (no retraining n times).

Decision Trees

- Decision trees are built iteratively, starting from a root that is a single attribute. Choosing the attribute to split on is chosen according to which one maximises some chosen metric: information gain or Gini impurity usually. Future splits continue in this fashion, until there are no features or all data points beyond a certain point belong to the same class.

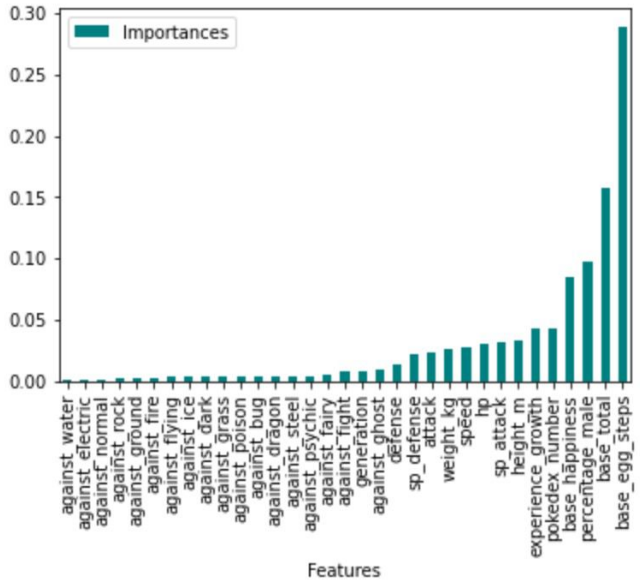


- This concept should be familiar: we saw this in the filter methods where mutual information (aka information gain) is used to determine which features show (in)dependence and which are the most important. Decision trees calculate these importances as part of the algorithm! Decision trees are sensitive to root attributes!
- Random Forests use bagging to generalise decision trees
    - N decision trees are constructed by taking a bootstrap sample (sampling with replacement) of the data
    - Entropy (or information gain) is used to determine the feature splits at each node, however, at each node, a random subset of F features is used as the candidate list for splitting
    - In each individual tree, features are ranked according to how much the entropy is decreased each time they are used for splitting at a node, weighted by the number of observations at that node.
    - The overall importance for each feature is calculated as an average of their importances in each individual tree. Similarly, the final prediction from the random forest is an average (or majority vote) of all N tree predictions
    - These are also called ensemble models

Random Forest Importances

```
model = RandomForestClassifier(n_estimators=250, random_state=42, max_depth=10)
# fit the model to start training.
model.fit(x_train, ds[d]['y_train'])
# get the importance of the resulting features.
importances = model.feature_importances_
# create a data frame for visualization.
final_df = pd.DataFrame({ "Features": pd.DataFrame(x_train).columns, "Importances": importances})
final_df.set_index('Importances')
# sort in ascending order to better visualization.
final_df = final_df.sort_values('Importances')
# plot the feature importances in bars.
final_df.plot.bar(color='teal', x='Features')
print(final_df)
```

- There are plenty of variables that don't look important – we should remove a lot of them.
- This is a hyperparameter itself



LASSO

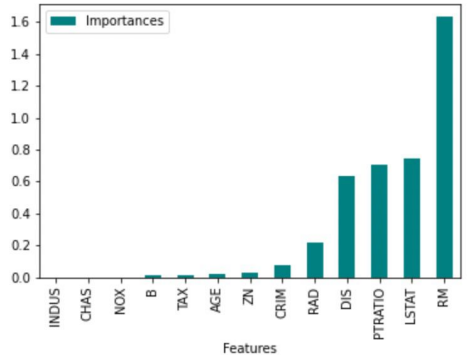
- A linear model assumes a relationship between the target variable, y, and a set of n input features, **x**:
  - $y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots \beta_n x_{in} + \epsilon_i$, where $\boldsymbol{\beta}$ is a set of coefficients that represent relevant weight of each feature x, and ϵ is a noise/intercept term. The fitting can be done by minimising the sum of squares error:
  - $SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i)$, where $\hat{y}_i$ is the predicted value from the model
- The weights, β, can be seen as a type of feature importance for each of the the input features, however, in general, these can become large if there is overfitting. We can control this to some extent by adding a penalty term to the weights (usually L2). While L2 penalties shrink the parameter estimates towards 0, it does not set the values to absolute 0 for any values of β. However, using LASSO (also known as L1 norm), we add a penalty to the absolute value of the weights
  - $SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i) + \lambda \sum_{j=1}^{P} |\beta_j|$, where λ is the shrinking penalty
  - The practical implication of this is that some of the parameters are set to 0, essentially conducting feature selection
- Models penalised with the L1 norm have sparse solutions: many of the fitted coefficients are zero
- For classification, an extension of this can be applied to logistic regression or LinearSVC, where an L1 penalty is also applied
- See SKLearn implementations of LASSO

LASSO Importances

```python
from sklearn.linear_model import Lasso
import numpy as np
dataset = 'boston'
# Set the regularization parameter C=1
lasso = Lasso(random_state=7)
lasso.fit(ds[dataset]['x_train'], ds[dataset]['y_train'])
# coef_lasso_ = np.array([lasso.coef_ for y in ds[dataset]['y_train'].T])
importances = np.abs(lasso.coef_)
final_df = pd.DataFrame({ "Features": pd.DataFrame(ds[dataset]['x_train']).columns, "Importances": importances})
final_df.set_index('Importances')
# sort in ascending order to better visualization.
final_df = final_df.sort_values('Importances')
# plot the feature importances in bars.
final_df.plot.bar(color='teal', x='Features')
print(final_df)
# now we just have to choose which ones to keep... top 5? Top 3?
```

Again, we have only about 4 variables that are really important

	Features	Importances
2	INDUS	0.000000
3	CHAS	0.000000
4	NOX	0.000000
11	B	0.011181
9	TAX	0.012286
6	AGE	0.016395
1	ZN	0.028501
0	CRIM	0.076609
8	RAD	0.219654
7	DIS	0.630858
10	PTRATIO	0.708582
12	LSTAT	0.747107
5	RM	1.630489



SelectFromModel (Automatic Selection)

- Sklearn has some useful functionality for helping to choose features!
  - SelectFromModel: Tell the algorithm which learning algorithm to run, and give it some parameters, then it will choose the best performing features for you

```
dataset = 'breast_cancer'
# Set the regularization parameter C=1
logistic = LogisticRegression(C=1, penalty="l1", solver='liblinear', random_state=7)
model = SelectFromModel(logistic)  # this chooses the best features for us!
model.fit(ds[dataset]['x_train'], ds[dataset]['y_train'])
# model.

selected_feat = ds[dataset]['x_train'].columns[(model.get_support())]
print('total features: {}'.format((ds[dataset]['x_train'].shape[1])))
print('selected features: {}'.format(len(selected_feat)))
print('features with coefficients shrank to zero: {}'.format(
      np.sum(model.estimator_.coef_ == 0)))
print(selected_feat)
```

```
total features: 30
selected features: 10
features with coefficients shrank to zero: 20
Index(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
       'texture error', 'area error', 'worst texture', 'worst perimeter',
       'worst area', 'worst concavity'],
      dtype='object')
```

Feature Stability & Cross Validation

- Do the features offer similar performance on variations of the dataset? You can test this using k-fold cross-validation!
  - When looping through the features in your search procedure, you can run this over multiple folds of the data too
- Use ensemble methods. Use multiple feature selectors, aggregate and chosen from this (frequency or weighted voting)
  - This is essentially what a Random Forest does, but you can do it with other algorithms too, and weight the outcome from each

Useful Python modules

- Mlxtend that has a bunch of useful stuff in it that will run the different search procedures with your choice of learning algorithm and the scoring function for the search
- Scikit-learn has many feature selection methods implemented separately and within the models.
- Attached jupyter notebooks have implementations of the discussed methods.

Lastly

- **There is no best feature selection method!**
- Highly depends on data.
- *If you have domain knowledge or a subject matter expert:* Select features based on expertise.
- Features needs to be on the same scale. Normalise and standardise your features
- It is good to check linearity first.
- In practice, optimal feature selection may take iterations over different methods
- Cross validating feature set is very important!